Lines-VQA: How to Use It

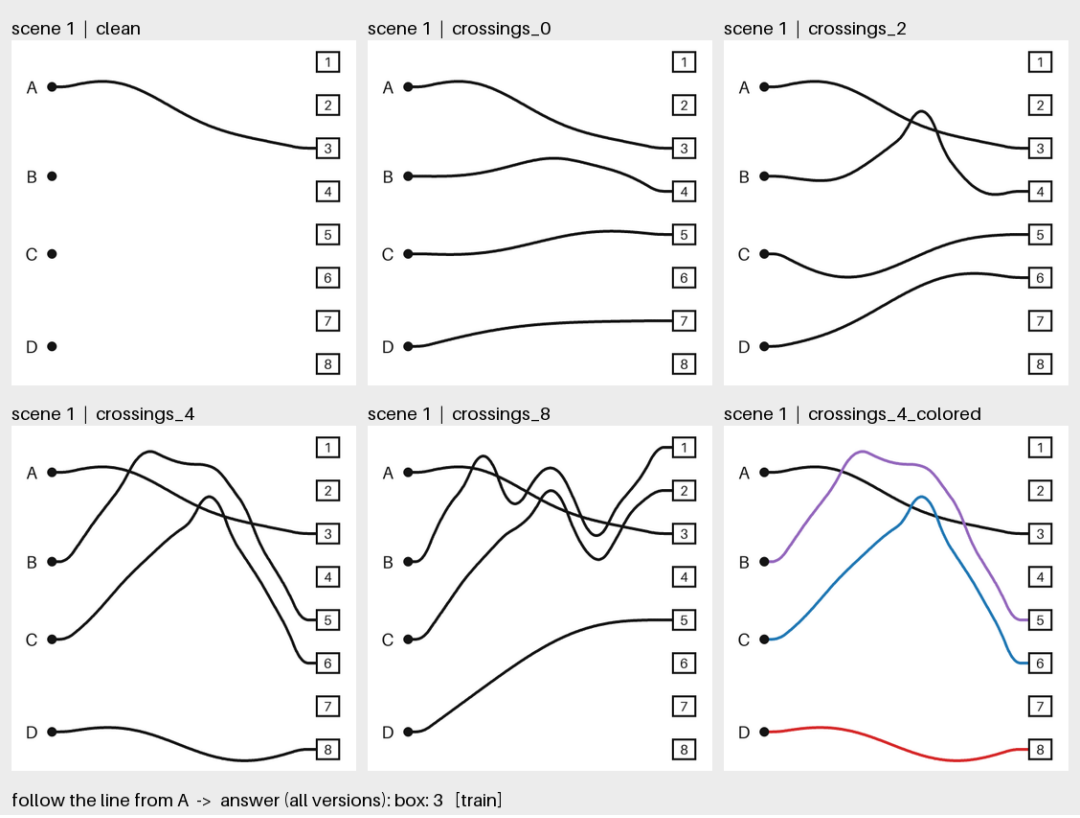
ATML project | huggingface.co/datasets/atml-shapes-vqa/lines-vqa | questions: ask Mohid

What it is

Our fourth dataset, and a different skill: **following a line** instead of counting. Four dots labelled **A-D** sit on the left, eight numbered **boxes 1-8** on the right, and lines run from dots to boxes. The question names one dot; the answer is the box its line ends in. Every place where another line crosses the asked-about line (the *target*) is a spot where the two are tangled together, so the **number of crossings is a direct dial for entanglement**. **500 scenes x 6 versions = 3,000 images** (512 x 512).

Version	Diff.	What is drawn besides the target line	What it tests
clean	0	nothing (the dots and boxes are always drawn)	baseline
crossings_0	1	lines from the 3 other dots; **none** crosses the target	plain clutter
crossings_2	2	the 3 lines cross the target **2** times in total	entanglement
crossings_4	3	... **4** times	entanglement
crossings_8	4	... **8** times	entanglement
crossings_4_colored	5	the **same lines** as crossings_4, but coloured (target stays black)	does separating the target help?

Key idea: the target line, the dots and the boxes are pixel-identical in all 6 versions of a scene, so **all 6 versions have the same correct answer**. crossings_0 to crossings_8 differ only in how often the other lines cross the target, so they give **accuracy vs. entanglement**. crossings_4 and crossings_4_colored have exactly the same lines; only the colour differs.



The question is the same for every image except the letter:

```
Follow the line that starts at the dot labelled B.
Which numbered box does it end in?
Lines may cross each other, but a line never turns onto another line at a crossing.
Answer in exactly this format: box: <number>

answer:  box: 5
```

Every crossing is a clean X (at least 45 degrees), lines never touch except where they cross, and the other lines never cross each other, so every crossing is on the target. Crossing counts are even because of geometry. The other lines may end in different boxes in different versions; the target never changes. Splits are by scene: **train** 2,100 images (350 scenes), **validation** 450 (75), **test** 450 (75).

Useful columns

Column	Meaning
`image_id`	e.g. line_0042_crossings_4. The key for your predictions
`scene_id`	same for all 6 versions of a scene. Use it to compare versions
`version, difficulty`	clean / crossings_0 / _2 / _4 / _8 / crossings_4_colored = 0-5
`target_dot, target_box`	where the target line starts and ends (target_box is the answer)
`num_crossings`	how often other lines cross the target (0, 2, 4 or 8)
`distractor_boxes, crossed_boxes`	boxes where the other lines end / the ones whose line crosses the target
`rank_kept`	yes if "the line goes straight across" would also give the right box (see below); empty for clean
`objects`	JSON: dots, boxes and every line (role target / distractor) with its control points. Useful for masks of the lines

Step 1. Get access (skip if you already did this for the other datasets)

1. Join the **atml-shapes-vqa** organization on Hugging Face (Mohid sends the invite link). 2. At **huggingface.co/settings/tokens** create a token of type Read. 3. In a terminal:

```
pip install datasets huggingface_hub pillow
hf auth login          # paste your token (older versions: huggingface-cli login)
```

Step 2. Load the data

```
from datasets import load_dataset
ds = load_dataset("atml-shapes-vqa/lines-vqa")
test = ds["test"]              # 450 images = 75 scenes x 6 versions
ex = test[0]
ex["image"]          # PIL image, 512 x 512
ex["target_dot"]     # e.g. "B"
ex["question"]       # the prompt (already names the dot)
ex["answer"]         # "box: 5"
ex["version"]        # clean / crossings_0 / _2 / _4 / _8 / crossings_4_colored
hard = test.filter(lambda e: e["version"] == "crossings_8")    # one version only
```

Step 3. Run your model and save its answers

Same as the other datasets: a CSV of `image_id, prediction`. Always pass the given question (it names the dot).

```
import csv
def ask_model(image, question):
    ...   # your VLM here (Qwen2-VL, LLaVA, InternVL, ...). Return its text answer.

with open("preds_lines_mymodel.csv", "w", newline="", encoding="utf-8") as f:
    w = csv.writer(f)
    w.writerow(["image_id", "prediction"])
    for ex in test:
        w.writerow([ex["image_id"], ask_model(ex["image"], ex["question"])])
```

Step 4. Score it

```
# once: download the scoring script (the same script works for all our datasets)
hf download atml-shapes-vqa/lines-vqa vqa_scoring.py --repo-type dataset --local-dir .

# score: it sees the "line_" image ids and fetches the right answers by itself
python vqa_scoring.py --predictions preds_lines_mymodel.csv --split test
```

You get one row per version: **acc** (right box), **jumped** (answered the box of a line that crosses the target, i.e. it switched lines at a crossing), **other** (box of any other line), **no_line** (a box no line ends in), **parsed**, and accuracy split by **rank_kept**, plus a **paired comparison against clean** with a McNemar p-value. The parser accepts box: 5, Box 5, box five or just 5.

How to read the results

- Drop from `clean` to `crossings_0`: plain clutter hurts, even without a single crossing.
- `crossings_0` to `crossings_8`: plot accuracy against the number of crossings. A steady fall means entanglement hurts, since nothing else changes.
- High **jumped**: the model loses the line at a crossing and follows the other one.
- `crossings_4_colored` vs. `crossings_4`: same lines, only the colour differs. If accuracy comes back, making the target easy to separate helps, which is what our disentanglement idea predicts.
- **rank_kept**: in about 70% of the crossing images (and all of `crossings_0`) the target ends in the same position among the line ends as it starts among the dots, so guessing "straight across" is right there. Compare *acc if rank kept* with *acc if changed*: a model that really traces scores about the same on both; a big gap means it is guessing.

Rules of thumb

- **Report results on the test split**; use train / validation for anything you tune. **Use the question as given**, so the 6 versions stay comparable. If something looks wrong, tell Mohid.